

ABC RETAIL SOLUTIONS

Data Engineering Pipeline

PROJECT DOCUMENTATION

Platform	Microsoft Fabric (Trial)
Language	PySpark / SQL
Storage	Delta Lake (OneLake)
Dashboard	Power BI (DirectLake)
Client	ABC Retail Solutions
Assessment	NeoStats Data Engineering Internship

Table of Contents

❖ Executive Summary

❖ Architecture

- Diagram
- Medalion Architecture
- Tech Stack

❖ Data Flow Diagram

❖ Data Model

❖ Assumptions Made

❖ Pipeline Stages

- Stage 1: Ingestion
- Stage 2: Transformation and Cleaning
- Stage 3: PII Masking
- Stage 4: Enrichment
- Stage 5: Revenue Calculations
- Stage 6: Aggregation (Gold tables)

❖ Power BI Stage

- KPIs
- Visual Insights

❖ Notebook Cell Reference

❖ Scalability and Production Considerations

❖ GitHub Repository Structure

Executive Summary:

ABC Retail Solutions, a multinational retail company, operates across multiple cities through both online and offline sales channels. The organization collects transactional retail data from various systems; however, inconsistencies in the dataset such as duplicate transactions, missing values, inconsistent product naming conventions, invalid quantities, and varying date formats have created challenges in generating accurate business insights. Without a standardized and reliable data pipeline, the company faces difficulties in analysing revenue trends, identifying bestselling products, monitoring category performance, and making data-driven business decisions.

This project delivers a complete end-to-end data engineering pipeline built entirely within Microsoft Fabric. The pipeline ingests raw Excel files into a Lakehouse, applies a comprehensive cleaning and transformation workflow using PySpark, masks Personally Identifiable Information using SHA-256 hashing, and produces five pre-aggregated Gold Delta tables that power an interactive Power BI dashboard. The total pipeline processes 8,401 transactions and yields a verified total revenue of 1.24 billion across four product categories and five cities.

Pipeline Summary:

```
=====
PIPELINE COMPLETE – FINAL SUMMARY
=====
Total Revenue      : ₹1,239,132,235.00
Total Transactions  :           8,401
Average Order Value : ₹    147,498.18
Silver table rows   :           8,401
Gold tables written :      5
=====
```

Delta Tables in Lakehouse:

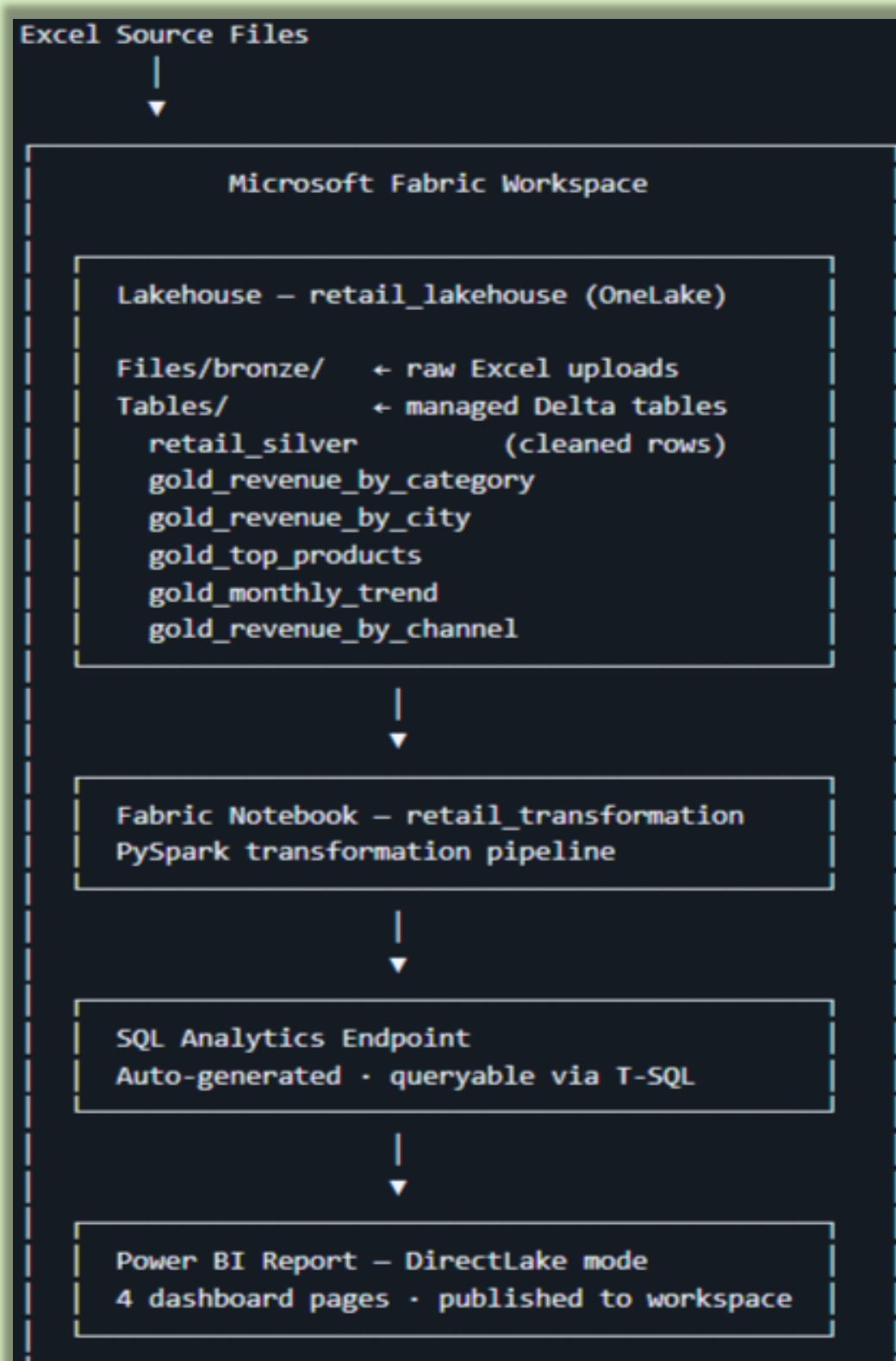
namespace	tableName	isTemporary
`ABC-Retail-Analytics`.retail_lakehouse.dbo	gold_monthly_trend	false
`ABC-Retail-Analytics`.retail_lakehouse.dbo	gold_revenue_by_category	false
`ABC-Retail-Analytics`.retail_lakehouse.dbo	gold_revenue_by_city	false
`ABC-Retail-Analytics`.retail_lakehouse.dbo	gold_top_products	false
`ABC-Retail-Analytics`.retail_lakehouse.dbo	retail_silver	false

(retail_transformation_notebook.ipynb)

Architecture:

The solution uses an entirely Fabric-native architecture. Every component such as storage, compute, SQL serving, and reporting, lives within a single Microsoft Fabric workspace.

1. Diagram: (Github Screenshot)



2. Medalion Architecture:

Zone	Location	Description
Bronze	Files/bronze/	Raw Excel files exactly as received. No transformations applied. Serves as the immutable source of truth.
Silver	Tables/retail_silver	Cleaned, deduplicated, PII-masked, enriched, and revenue-calculated dataset. One row per valid transaction.
Gold	Tables/gold_*	Pre-aggregated KPI tables grouped by category, city, product, month, and sales channel. Directly consumed by Power BI.

(Claude Generated)

3. Tech Stack:

Layer	Tool	Purpose
Platform	Microsoft Fabric (Trial)	Unified analytics workspace
Storage	Fabric Lakehouse (OneLake)	Bronze / Silver / Gold zones
Processing	Fabric Notebook (PySpark)	Data transformation pipeline
File format	Delta Lake	Versioned, queryable tables
SQL layer	SQL Analytics Endpoint	Ad-hoc queries, semantic model
Reporting	Power BI (DirectLake)	Dashboard, KPIs, interactivity
Language	Python 3 / PySpark	Transformation logic
PII protection	SHA-256 hashing	One-way irreversible masking

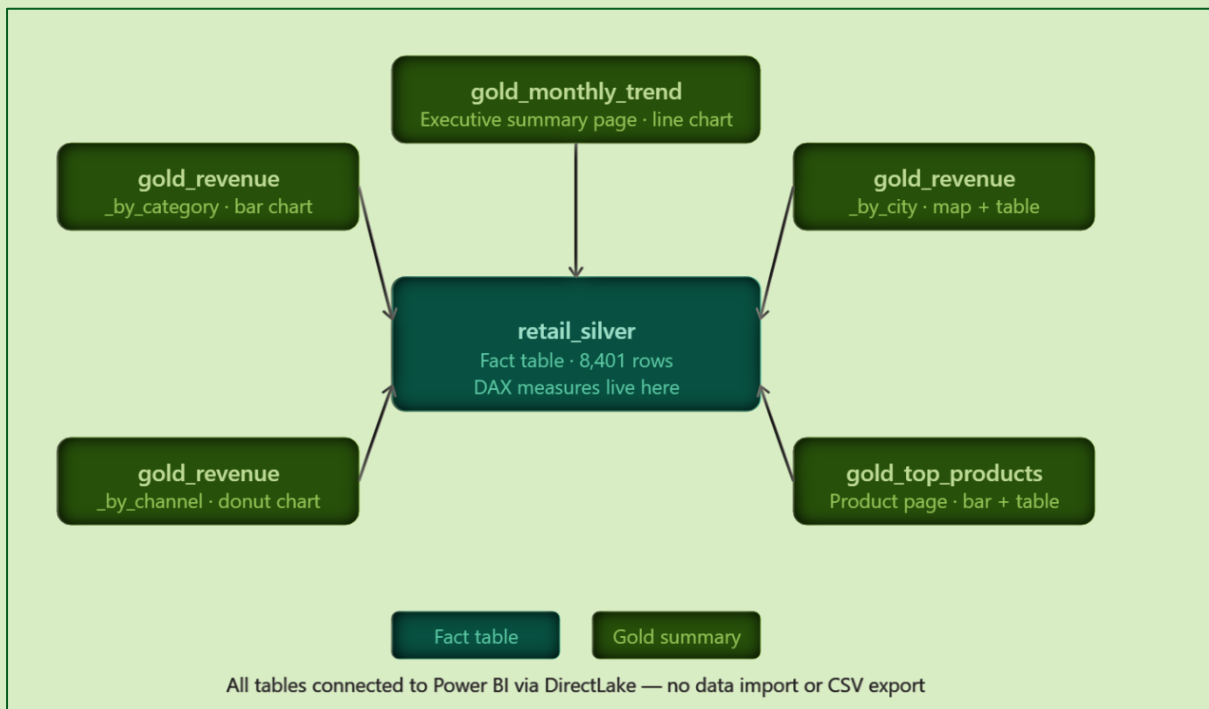
(From Github README.md, Screenshot)

Data Flow Diagram:

```
[retail_data1.xlsx]  [retail_data2.xlsx]  [product_details.xlsx]
|
+-----+
|
| pandas read_excel()
|
| spark.createDataFrame()
|
| unionByName()  <-- stacks both sources
|
| dropDuplicates()
|
| fillna() + filter(qty > 0)
|
| initcap(trim()) + date parsing
|
| category mapping (Elec->Electronics etc.)
|
| sha2(email, 256) + sha2(phone, 256)
|
+----- LEFT JOIN -----+
|           (on product_id)
|
| revenue = (price - discount*price) * qty
|
| filter(revenue >= 0)
|
| saveAsTable('retail_silver')  [Silver zone]
|
+-----+-----+-----+-----+
|           |           |           |           |
gold_      gold_      gold_      gold_      gold_
category   city      products  monthly   channel
|           |           |           |           |
+-----+-----+-----+-----+
|
| Power BI DirectLake (reads Delta live)
|
| [3-page interactive dashboard]
```

(Claude Generated)

Data Model:



(Claude generated)

Assumptions Made:

- ❖ Rows with quantity ≤ 0 are treated as returns or invalid entries and excluded from analysis.
- ❖ Missing city, category, and 'payment_method' values are filled with 'Unknown' rather than dropped, to preserve transaction revenue records.
- ❖ Where "price" is missing, the "standard_product_price" from the product dimension table is used as a substitute.
- ❖ Date strings in GregorianCalendar object format (YEAR=2024,MONTH=0,...) are parsed separately and merged with standard date formats.
- ❖ Category abbreviations ("Elec", "Furn") are treated as data entry errors and mapped to their full standard names.
- ❖ The pipeline is designed to be idempotent — re-running it always produces the same output (Delta tables use overwrite mode).

Pipeline Stages:

(from [github](#))

Stage 1 Ingestion

Raw Excel files are uploaded manually into the Files/bronze/ folder of the Fabric Lakehouse. This simulates a production ingestion layer where files would arrive from source systems automatically. Files are read into PySpark DataFrames using pandas (for reliable .xlsx support) and then converted to Spark DataFrames via `spark.createDataFrame()`.

Stage 2 Transformation and Cleaning

The following cleaning operations are applied in sequence:

Issue	Resolution
Inconsistent column names	Stripped, lowercased, spaces replaced with underscores
Duplicate rows	Removed using <code>dropDuplicates()</code>
Missing categorical values	Filled with 'Unknown' (city, category, payment_method)
Missing customer info	Filled with 'Not Provided'
Invalid quantity (≤ 0)	Rows removed — represent returns or data entry errors
Inconsistent text casing	Standardized to Title Case using <code>initcap(trim())</code>
Abbreviated category values	Mapped to full names (e.g. "Elec" → "Electronics")
Mixed date formats	Parsed using <code>coalesce()</code> across multiple <code>to_date()</code> patterns
GregorianCalendar date strings	Extracted with <code>regexp_extract()</code> and reconstructed with <code>make_date()</code>

Stage 3 PII Masking

Customer email and phone columns are masked using **SHA-256 hashing** via PySpark's built-in `sha2(col, 256)` function. The original columns are dropped and replaced with `email_masked` and `phone_masked`. SHA-256 was chosen over simple redaction because it is:

- **One-way** — cannot be reversed to recover the original value
- **Consistent** — the same input always produces the same hash (useful for counting unique customers without exposing PII)
- **Standards-compliant** — satisfies data protection requirements including India's DPDP Act

Stage 4 Enrichment

The cleaned transaction data is left-joined with the `product_details` dimension table on `product_id`. This adds standardized product names and category information. Missing unit prices are back-filled from the `standard_product_price` column in the dimension table.

Stage 5 — Revenue Calculation

$\text{revenue} = (\text{price} - \text{discount} * \text{price}) \times \text{quantity}$

Rows with negative revenue (returns or errors) are removed after calculation.

Stage 6 — Aggregation (Gold tables)

Five Gold Delta tables are written for direct Power BI consumption:

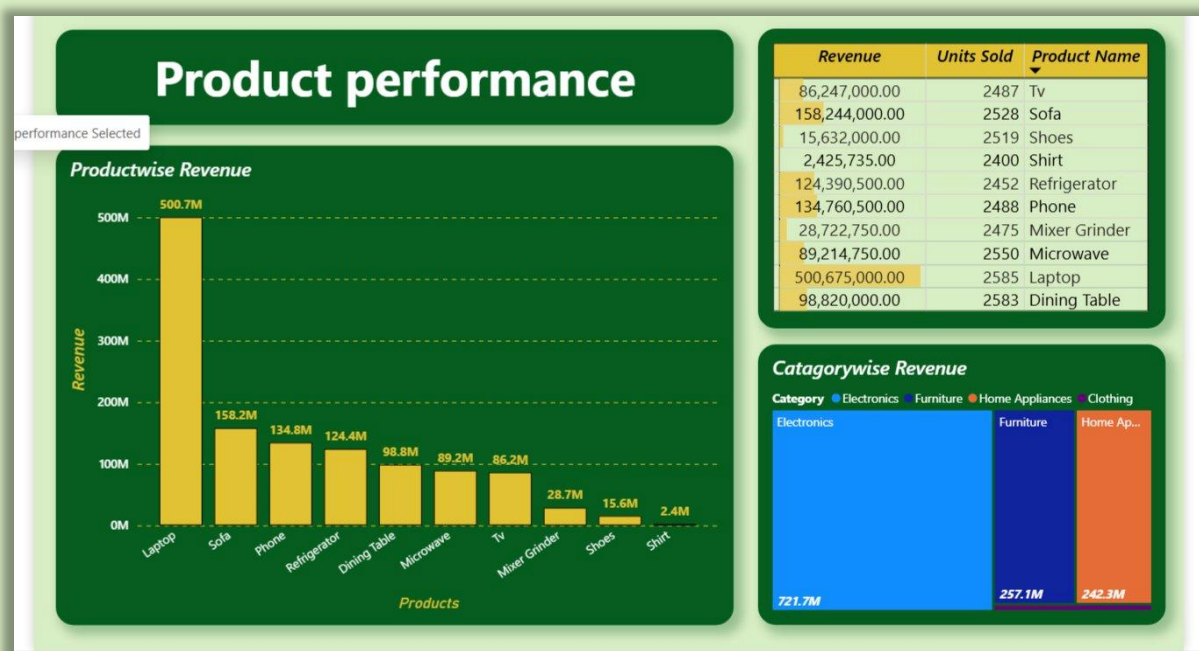
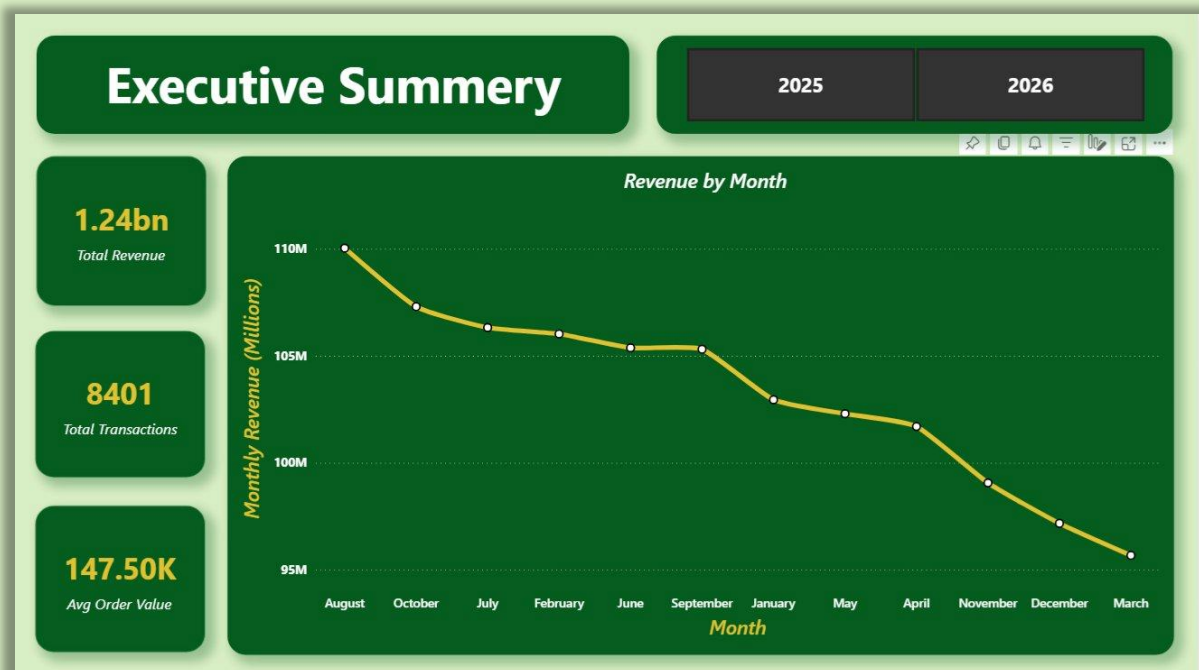
Table	Grain	Key Metrics
<code>gold_revenue_by_category</code>	Category	<code>total_revenue</code> , <code>transaction_count</code> , <code>avg_order_value</code>
<code>gold_revenue_by_city</code>	City	<code>total_revenue</code> , <code>transaction_count</code> , <code>unique_customers</code>
<code>gold_top_products</code>	Product	<code>units_sold</code> , <code>total_revenue</code>
<code>gold_monthly_trend</code>	Year + Month	<code>monthly_revenue</code> , <code>transaction_count</code>
<code>gold_revenue_by_channel</code>	Sales channel	<code>total_revenue</code> , <code>transaction_count</code>

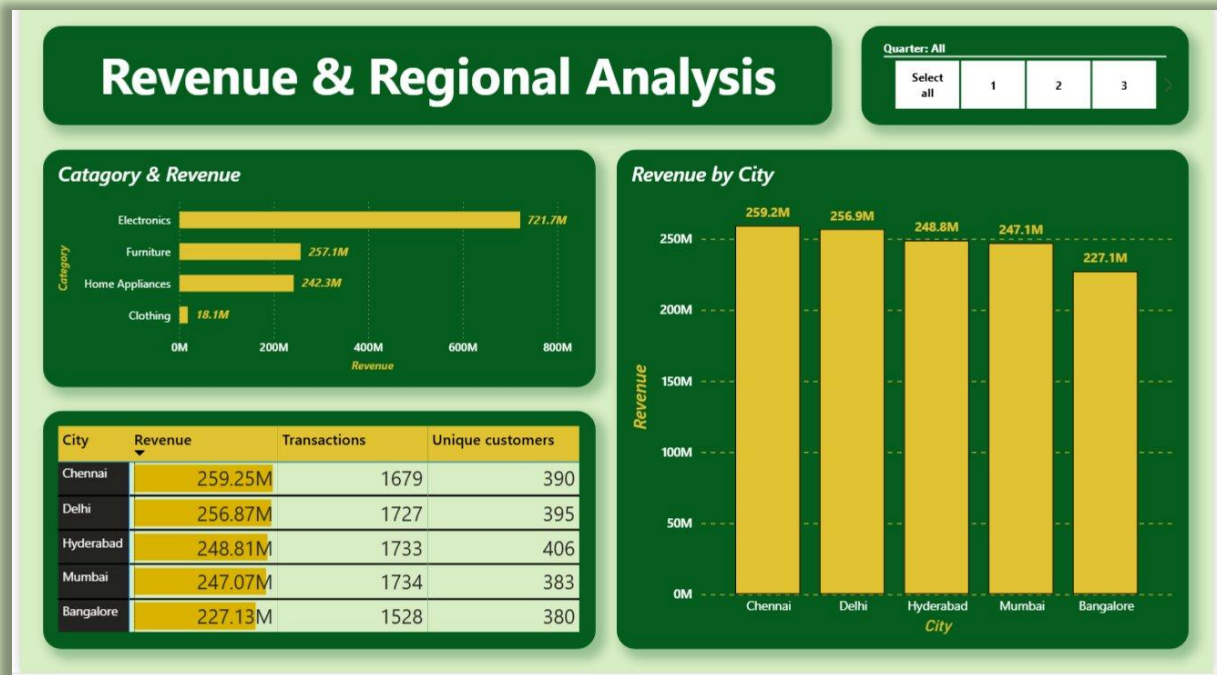
Power BI Stage:

KPIs:

- ❖ Total Revenue = SUM(retail_silver[revenue])
- ❖ Total Transactions = COUNTROWS(retail_silver)
- ❖ Avg Order Value = DIVIDE([Total Revenue], [Total Transactions], 0)
- ❖ Revenue MoM % = VAR cur = [Total Revenue]
VAR prev = CALCULATE([Total Revenue],
DATEADD(retail_silver[transaction_date], -1, MONTH))
RETURN DIVIDE(cur - prev, prev, 0)

Visuals:





Notebook Cell Reference:

(from Github)

Cell	Purpose
Cell 1	Read Bronze Excel files using pandas
Cell 2	Convert pandas → Spark DataFrames
Cell 3	Standardize column names
Cell 4	Union retail_data1 and retail_data2
Cell 5	Remove duplicate rows
Cell 6	Handle missing values and invalid quantities
Cell 7	Standardize text, parse mixed date formats
Cell 7b	Standardize category abbreviations
Cell 8	PII masking — SHA-256 hash email and phone

Cell	Purpose
Cell 9	Enrich with product dimension table
Cell 10	Calculate revenue column
Cell 11	Write Silver Delta table
Cell 12	Write all Gold aggregation tables
Cell 13	Final summary and validation printout

GitHub Repository Structure:

```

ABC-Retail-Data-Pipeline/
├── Code/
│   └── retail_pipeline.py    # Python script exported from notebook
├── Notebooks/
│   └── retail_transformation_notebook    # Fabric Notebook (.ipynb export)
├── Data/
│   └── raw/
│       ├── retail_data1.xlsx
│       ├── retail_data2.xlsx
│       └── product_details.xlsx
├── Documentation/
│   └── Project_Documentation.docx      # Architecture, DFD, assumptions, logic
├── PowerBI/
│   ├── Retail_Dashboard.pbix
│   ├── page1_executive_summary.png
│   ├── page2_revenue & regional analysis.png
│   └── page3_product_performance.png
└── README.md

```